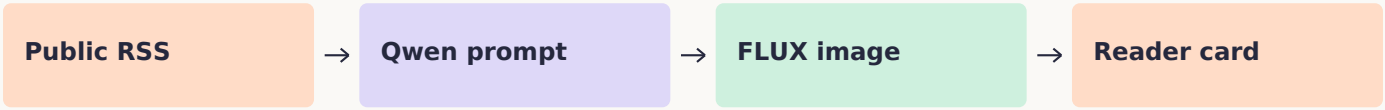
PROJECT NOTE / 20 OCT 2026

PulseFeed

An RSS reader that turns RSS stories into illustrated cards.

Paste a public RSS or Atom URL and click **View RSS**. PulseFeed uses each story's title and available excerpt to make a visual prompt, then an image. The reader keeps the source text and links, and clearly labels AI illustrations. It gives a feed a visual layer without pretending the generated pictures are news photographs.

The small reader is also a practical exercise in running two models: control what gets admitted, keep track of accepted jobs, and avoid paying for another image when someone refreshes.



FastAPI coordinates both stages. The workers never call each other.

Practical choices

Two separate workers. Qwen uses nvidia/Qwen3-8B-NVFP4; FLUX uses black-forest-labs/FLUX.2-klein-4B. Separate containers isolate their inference dependencies. GPU workers keep no durable application state; the app owns orchestration.

Small stateful app. One FastAPI process and a coordinator share persistent SQLite for admission, reservations, job IDs, usage and recovery. This is deliberately a single-instance design.

Temporary images. Reader pages and browser IndexedDB image Blobs expire after one hour. SQLite stores metadata, not image bytes. There is no S3 bucket or permanent image archive. Runpod results last 30 minutes, so a late return without a browser copy can leave an image unavailable.

Safeguards

Validate the boundaries. Public RSS destinations, DNS and redirects are checked; feed sizes and fetch time are bounded. Provider credentials stay on the server. Admin access requires authentication and CSRF protection.

Bound the spend. Global quotas reserve capacity before inference. New pages default to ten stories; the verification run used an operator override of nine. Refresh, polling and exact request replay never admit more work.

Treat uncertainty carefully. An explicit retry is allowed only after a known outcome and a new admission check. Unknown submission outcomes hold reservations and pause new dispatch for reconciliation, instead of automatically resubmitting a possibly accepted job.

Real proof

The controlled local-app-to-Runpod run produced **ten 512-square images from twenty successful model calls**, with no retries or duplicate invocations. Aggregate tokens and image attempts matched the saved results. Earlier Spark testing produced 27 images across Hacker News, BBC England and NYTimes.

Current status

Verified: native Spark inference, live Runpod model calls from the laptop, and Railway web ingress, authentication and persistence. **Next:** refresh Qwen’s CA bundle, redeploy the latest app and test the connected Railway generation flow. Cold starts dominate current latency; active workers are the first performance improvement. Remaining upstream dependency findings are documented.

Evidence and tradeoffs: docs/architecture.md and docs/dependency-remediation-2026-10-20.md.